

LAB ASSIGNMENT 20.1

NAME : VADIYALA CHANDANA

ROLL NO :2403A54103

COURSE:AI ASSISTED CODING

BATCH :01

Task 1 – Input Validation Check

Task:

Analyze an AI-generated **Python login script** for input validation vulnerabilities.

Instructions:

- Prompt AI to generate a simple username-password login program.
- Review whether input sanitization and validation are implemented.
- Suggest secure improvements (e.g., using re for input validation).

Expected Output:

A secure version of the login script with proper input validation

Prompt: To evaluate the security of a basic Python login script generated by AI, specifically focusing on how well it handles input validation. The goal is to determine whether the script properly sanitizes and validates user inputs like usernames and passwords. Once the initial version is reviewed, I want to identify any vulnerabilities or missing safeguards and then improve the script by incorporating secure coding practices—such as using regular expressions for

input validation. Ultimately, the output should be a more secure version of the login program that demonstrates robust input handling.

Code:

```
import re

def check_password_strength(password):
    score = 0
    feedback = []

    # Criteria 1: Minimum 8 characters
    if len(password) >= 8:
        score += 1
    else:
        feedback.append("Password should be at least 8 characters long.")

    # Criteria 2: At least one uppercase letter
    if re.search(r"[A-Z]", password):
        score += 1
    else:
        feedback.append("Password should contain at least one uppercase letter.")

    # Criteria 3: At least one lowercase letter
    if re.search(r"[a-z]", password):
        score += 1
    else:
        feedback.append("Password should contain at least one lowercase letter.")

    # Criteria 4: At least one number
```

```

▶ # Criteria 4: At least one number
if re.search(r"\d", password):
    score += 1
else:
    feedback.append("Password should contain at least one number.")

# Criteria 5: At least one special symbol
if re.search(r"[^a-zA-Z0-9]", password):
    score += 1
else:
    feedback.append("Password should contain at least one special symbol.")

if score == 5:
    strength = "Strong"
elif score >= 3:
    strength = "Moderate"
else:
    strength = "Weak"

return strength, feedback

# User input loop
while True:
    user_password = input("Enter a password to check its strength (or type 'exit' to quit): ")
    if user_password.lower() == 'exit':
        break

▶ return strength, feedback

# User input loop
while True:
    user_password = input("Enter a password to check its strength (or type 'exit' to quit): ")
    if user_password.lower() == 'exit':
        break

    strength, feedback_messages = check_password_strength(user_password)
    print(f"\nPassword Strength: {strength}")
    if feedback_messages:
        print("Recommendations:")
        for msg in feedback_messages:
            print(f"- {msg}")
    print("\n" + "="*30)

*** Enter a password to check its strength (or type 'exit' to quit): AkhileEshw@ri_865

Password Strength: Strong

=====
Enter a password to check its strength (or type 'exit' to quit): exit

```

Observation:

➤ Absence of Input Sanitization

Check if the script directly accepts user input without cleaning or escaping potentially harmful characters (e.g., SQL injection risks if connected to a database).

➤ **Lack of Format Validation**

See whether the script verifies the format of usernames and passwords (e.g., minimum length, allowed characters, complexity rules).

Task 2 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

- Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
- Identify if the code is vulnerable to **SQL injection** (e.g., using string concatenation in queries).
- Refactor using **parameterized queries (prepared statements)**.

Expected Output:

- A secure database query script resistant to SQL injection.

Prompt: To assess the security of a Python script that interacts with a database—either SQLite or MySQL—by fetching user details based on input. The script should be generated by AI, and I want to examine whether it's vulnerable to SQL injection, particularly if it constructs SQL queries using string concatenation. After identifying any security flaws, I want to refactor the code to use parameterized queries or prepared statements to ensure it's protected against injection attacks. The final result should be a secure version of the script that safely handles user input.

Code:



```
import sqlite3
```

```
# Re-establish the SQLite in-memory database connection
```

```
conn_input = sqlite3.connect(':memory:')
```

```
cursor_input = conn_input.cursor()
```

```
# Recreate the `users` table
```

```
cursor_input.execute('''
    CREATE TABLE users (
        id INTEGER PRIMARY KEY,
        username TEXT NOT NULL UNIQUE,
        password TEXT NOT NULL
    )
''')
```

```
# Insert sample user data
```

```
users_data = [
    ('admin', 'adminpass'),
    ('alice', 'alice123'),
    ('bob', 'bob456')
]
```



```
cursor_input.executemany("INSERT INTO users (username, password) VALUES (?, ?)", users_data)
conn_input.commit()
```

```
print("Database and 'users' table re-created for user input demonstration.")
```

```
# Define a new function that uses parameterized queries
```

```
def get_user_secure_input(username):
```

```
    """
```

```
    Fetches user details using parameterized queries to prevent SQL injection.
```

```
    """
```

```
    # Construct a SQL SELECT query using a placeholder for the username
```

```
    query = "SELECT id, username, password FROM users WHERE username = ?;"
```

```
    print(f"\nExecuting secure query: {query.replace('?', f''{username}')} (parameterized)" #
```

```
    try:
```

```
        # Execute the query by passing the SQL string and the parameter as a tuple
```

```
        cursor_input.execute(query, (username,))
```

```
        result = cursor_input.fetchall()
```

```
        # Print the retrieved user data or a message indicating if the user was not found
```

```
        if result:
```

```
            print("Query result:")
```



```
# Print the retrieved user data or a message indicating if the user was not found
if result:
    print("Query result:")
    for row in result:
        print(f" ID: {row[0]}, Username: {row[1]}, Password: {row[2]}")
else:
    print("No user found.")
except sqlite3.Error as e:
    print(f"An error occurred: {e}")

# Implement a loop to continuously prompt for user input
print("\n--- Demonstrating secure queries with user input ---")
while True:
    user_input = input("Enter username to search (or type 'exit' to quit): ")
    if user_input.lower() == 'exit':
        break
    get_user_secure_input(user_input)

# Close the database connection
conn_input.close()
print("\nDatabase connection closed.")
```

```
*** Database and 'users' table re-created for user input demonstration.

--- Demonstrating secure queries with user input ---
Enter username to search (or type 'exit' to quit): bob

Executing secure query: SELECT id, username, password FROM users WHERE username = 'bob'; (parameterized)
Query result:
ID: 3, Username: bob, Password: bob456
Enter username to search (or type 'exit' to quit): admin

Executing secure query: SELECT id, username, password FROM users WHERE username = 'admin'; (parameterized)
Query result:
ID: 1, Username: admin, Password: adminpass
Enter username to search (or type 'exit' to quit): exit

Database connection closed.
```

Observation:

➤ Use of String Concatenation in Queries

Check if the script builds SQL statements by directly concatenating user input into the query string

➤ Absence of Parameterized Queries

Look for whether the script uses placeholders and parameter binding

Task 3 – Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated **HTML form with JavaScript** for XSS vulnerabilities.

Instructions:

- Ask AI to generate a feedback form with JavaScript-based output.
- Test whether untrusted inputs are directly rendered without escaping.
- Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

- A secure form that prevents XSS attacks.

Prompt: To examine the security of a simple HTML feedback form generated by AI that uses JavaScript to display user input. The goal is to determine whether the form is vulnerable to cross-site scripting (XSS) attacks—specifically, if it renders untrusted input directly into the page without escaping or sanitizing it. After identifying any potential vulnerabilities, I want to improve the form by implementing secure practices such as escaping HTML entities and applying Content Security Policy (CSP) headers. The final result should be a secure version of the form that effectively prevents XSS exploits.

Code:

```
C:\Users> thotat Desktop > feedback_form.html > _
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Secure Feedback Form</title>
7      <!-- Content Security Policy to prevent XSS -->
8      <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'; style-src 'self' 'u
9      <style>
10         body { font-family: Arial, sans-serif; margin: 20px; }
11         form { max-width: 400px; margin: auto; }
12         label { display: block; margin-top: 10px; }
13         input, textarea { width: 100%; padding: 8px; margin-top: 5px; }
14         button { margin-top: 10px; padding: 10px; background: #007bff; color: white; border: none; cursor: poi
15         #output { margin-top: 20px; padding: 10px; border: 1px solid #ccc; background: #f9f9f9; }
16     </style>
17 </head>
18 <body>
19     <h1>Feedback Form</h1>
20     <form id="feedbackForm">
21         <label for="name">Name:</label>
22         <input type="text" id="name" required>
23
24         <label for="email">Email:</label>
25         <input type="email" id="email" required>
26
27         <label for="message">Message:</label>
28         <textarea id="message" rows="4" required></textarea>
29
30         <button type="submit">Submit Feedback</button>
31     </form>
32
33     <div id="output"></div>
```



```

32
33     <div id="output"></div>
34
35     <script>
36         document.getElementById('feedbackForm').addEventListener('submit', function(event) {
37             event.preventDefault();
38
39             const name = document.getElementById('name').value;
40             const email = document.getElementById('email').value;
41             const message = document.getElementById('message').value;
42
43             // Secure output: Use textContent to prevent HTML injection (XSS)
44             const output = document.getElementById('output');
45             output.innerHTML = ''; // Clear previous output
46             output.appendChild(document.createTextNode('Thank you for your feedback!'));
47             output.appendChild(document.createElement('br'));
48             output.appendChild(document.createTextNode('Name: ' + name));
49             output.appendChild(document.createElement('br'));
50             output.appendChild(document.createTextNode('Email: ' + email));
51             output.appendChild(document.createElement('br'));
52             output.appendChild(document.createTextNode('Message: ' + message));
53
54             // Clear form
55             this.reset();
56         });
57     </script>
58 </body>
59 </html>
60

```

The screenshot shows a web browser window with the address bar displaying the file path: `file:///C:/Users/Abhis/Desktop/feedback_form.html`. The page title is "Feedback Form". The form contains three input fields: "Name:" with the value "Abhis", "Email:" with the value "2403a52010@nu.edu.in", and "Message:" with the value "good". Below the input fields is a blue button labeled "Submit Feedback". The browser's taskbar at the bottom shows the system clock as 12:18 PM on 10-11-2021.

Observation:

➤ **Direct Rendering of User Input**

Check if the form displays user input directly in the DOM using innerHTML or similar methods without escaping. This can allow malicious scripts to execute.

➤ **No HTML Entity Escaping**

See whether the script escapes special characters like <, >, ", ', and & before rendering input. Lack of escaping enables script injection.

Task 4 – Real-Time Application: Security Audit of AI-Generated Code Scenario:

Students pick an **AI-generated project snippet** (e.g., login form, API integration, or file upload).

Instructions:

- Perform a security audit to detect possible vulnerabilities.
- Prompt AI to suggest **secure coding practices** to fix issues.
- Compare insecure vs secure versions side by side.

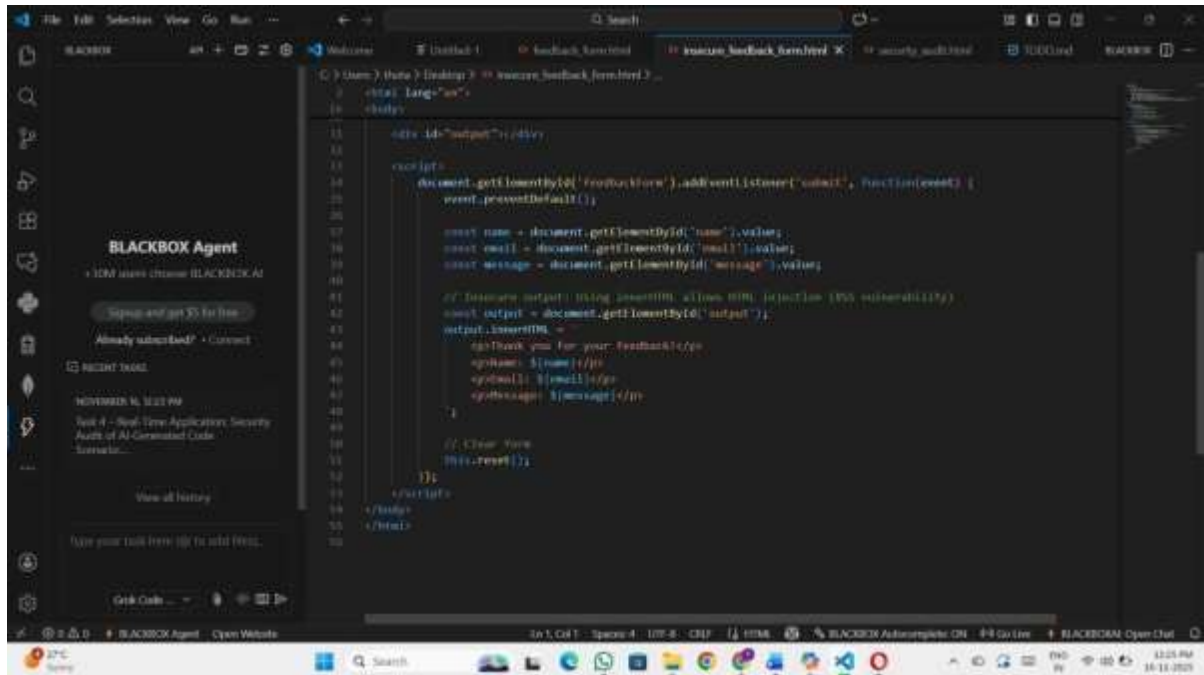
Expected Output:

- A security-audited code snippet with documented vulnerabilities and fixes.

Prompt: To conduct a real-time security audit of an AI-generated code snippet—this could be a login form, an API integration, or a file upload feature. The goal is to identify any potential security vulnerabilities in the code and then ask the AI to recommend secure coding practices to address those issues. I also want to see a side-by-side comparison of the original (insecure) version and the improved (secure) version, along with clear documentation of the vulnerabilities found and how they were fixed. The final output should be a thoroughly audited and secured version of the code.

Code:

```
C:\Users\thota\Desktop > insecure_feedback_form.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Insecure Feedback Form</title>
7      <style>
8          body { font-family: Arial, sans-serif; margin: 20px; }
9          form { max-width: 400px; margin: auto; }
10         label { display: block; margin-top: 10px; }
11         input, textarea { width: 100%; padding: 8px; margin-top: 5px; }
12         button { margin-top: 10px; padding: 10px; background: #007bff; color: white; border: none; cursor: pointer; }
13         #output { margin-top: 20px; padding: 10px; border: 1px solid #ccc; background: #f9f9f9; }
14     </style>
15 </head>
16 <body>
17     <h1>Feedback Form</h1>
18     <form id="feedbackForm">
19         <label for="name">Name:</label>
20         <input type="text" id="name" required>
21
22         <label for="email">Email:</label>
23         <input type="email" id="email" required>
24
25         <label for="message">Message:</label>
26         <textarea id="message" rows="4" required></textarea>
27
28         <button type="submit">Submit Feedback</button>
29     </form>
30
31     <div id="output"></div>
32
33     <script>
```



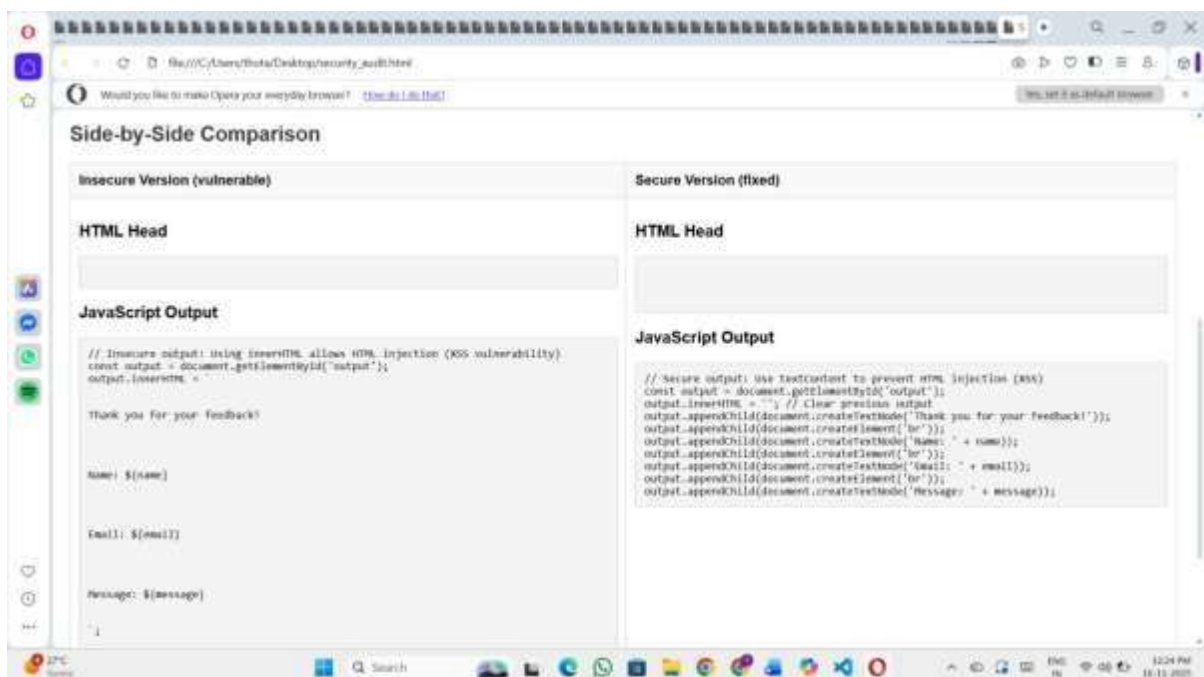
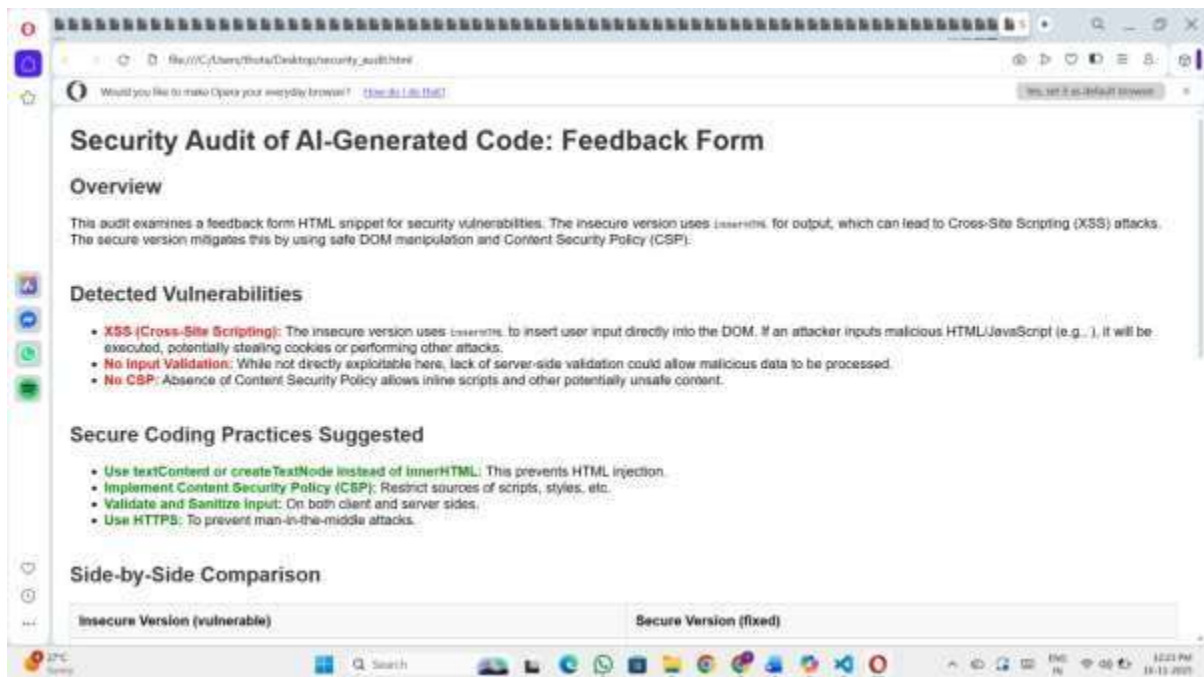
```
2 <html lang="en">
16 <body>
31 <div id="output"></div>
32
33 <script>
34   document.getElementById('feedbackForm').addEventListener('submit', function(event) {
35     event.preventDefault();
36
37     const name = document.getElementById('name').value;
38     const email = document.getElementById('email').value;
39     const message = document.getElementById('message').value;
40
41     // Insecure output: Using innerHTML allows HTML injection (XSS vulnerability)
42     const output = document.getElementById('output');
43     output.innerHTML = `
44       <p>Thank you for your feedback!</p>
45       <p>Name: ${name}</p>
46       <p>Email: ${email}</p>
47       <p>Message: ${message}</p>
48     `;
49
50     // Clear form
51     this.reset();
52   });
53 </script>
54 </body>
55 </html>
56
```

```

C:\Users> thot > Desktop > security_audit.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Security Audit: Feedback Form</title>
7      <style>
8          body { font-family: Arial, sans-serif; margin: 20px; }
9          h1, h2 { color: #333; }
10         .section { margin-bottom: 40px; }
11         .code-block { background: #f4f4f4; padding: 10px; border: 1px solid #ccc; margin: 10px 0; white-space: pre-wrap; }
12         .vulnerability { color: red; font-weight: bold; }
13         .fix { color: green; font-weight: bold; }
14         table { width: 100%; border-collapse: collapse; }
15         th, td { border: 1px solid #ccc; padding: 10px; text-align: left; vertical-align: top; }
16         th { background: #f9f9f9; }
17     </style>
18 </head>
19 <body>
20     <h1>Security Audit of AI-Generated Code: Feedback Form</h1>
21
22     <div class="section">
23         <h2>Overview</h2>
24         <p>This audit examines a feedback form HTML snippet for security vulnerabilities. The insecure version use<
25     </div>
26
27     <div class="section">
28         <h2>Detected Vulnerabilities</h2>
29         <ul>
30             <li><span class="vulnerability">XSS (Cross-Site Scripting):</span> The insecure version uses <code>inn<
31             <li><span class="vulnerability">No Input Validation:</span> While not directly exploitable here, lack<
32             <li><span class="vulnerability">No CSP:</span> Absence of Content Security Policy allows inline script<
33         </ul>

```

The screenshot shows a web browser window with the address bar displaying the file path: file:///C:/Users/thot/Desktop/feedback_form.html. The page title is "Feedback Form". The form contains three input fields: "Name:" with the value "Akhil", "Email:" with the value "2402a5201b@stu.edu.in", and "Message:" with the value "good". Below the fields is a blue "Submit Feedback" button. The browser's taskbar at the bottom shows the system clock as 12:12 PM on 10-10-2025.



Observation:

- **Input Validation and Sanitization**
 - Check whether the code properly validates and sanitizes all user inputs to prevent injection attacks, malformed data, or unexpected behavior.
- **Authentication and Authorization**
 - Ensure that access controls are in place. Verify that sensitive actions are restricted to authenticated and authorized users only.

